MINI PROJECT REPORT

AI-POWERED SMART CAMPUS HELPDESK & INCIDENT MANAGEMENT SYSTEM

MINI PROJECT REPORT – AGILE PLANNING & SCRUM IMPLEMENTATION

Student Name	Nikhil Balasaheb Shidore
Department	Artificial Intelligence & Data Science (AIDS)
PRN	2126UDSM2575
Subject	DevOps and Cloud Foundation (IBM)
Project Type	Agile Planning & Scrum Implementation

1. INTRODUCTION

Software product engineering is a structured process of identifying a user problem, understanding requirements, designing a solution, developing and testing the product, deploying it and improving it continuously. Coding is only one part of this process; the product must also be secure, maintainable, useful and aligned with user needs.

The AI-Powered Smart Campus Helpdesk & Incident Management System is proposed as a centralized platform for students, faculty, IT support staff and administrators. Users can report campus incidents such as Wi-Fi connection issues, lab computer faults, classroom projector problems and login difficulties. Support staff can assign tickets, set priorities, update progress and resolve incidents.

The project uses Agile planning and Scrum to divide development into manageable Sprints. The team delivers useful features incrementally, demonstrates the working product, collects feedback and adapts the backlog. The earlier Unit 1 assignment provides the product concept and proposed cloud architecture; the supplied Agile Planning & Scrum workbook provides the planning activities and Scrum deliverables.

2. PROBLEM STATEMENT AND BACKGROUND

In a manual support process, a student may visit the IT office, send an email or enter a complaint in a paper register. The report may not contain all the information required by support staff. It can be difficult to know who owns the issue, whether the issue is urgent, and when it will be resolved. Repeated questions about Wi-Fi or account access can also consume staff time.

A centralized helpdesk can reduce these problems by storing incidents in one place and giving each report a unique ticket number. A consistent workflow helps staff track ownership and status. A dashboard can show active, resolved and overdue tickets, while an AI-assisted FAQ feature can provide guidance for configured common questions.

2.1 Existing System Limitations

- Reports are spread across emails, paper registers and in-person conversations.
- Users may not receive consistent progress updates.
- Urgent incidents can be mixed with routine requests.
- Repeated common questions increase the workload of support staff.
- Manual counting makes it harder to monitor trends and overdue incidents.

2.2 Proposed Solution

- Provide one digital portal for incident reporting and status tracking.
- Generate a unique ticket ID and store the ticket details centrally.
- Use category and priority to support ticket assignment and routing.
- Provide an AI-assisted FAQ for configured support questions.
- Display ticket summaries through an administrator dashboard.

3. PRODUCT VISION, OBJECTIVES AND VALUE

Product Vision: For students, faculty, IT support staff and campus administrators, the AI-Powered Smart Campus Helpdesk provides a secure and convenient way to report incidents, receive guidance for common questions and track resolution progress. It reduces manual coordination, improves visibility and helps the support team prioritize urgent campus issues.

3.1 Objectives

- Create a simple and accessible incident-reporting process.
- Provide role-aware access to tickets and administrative functions.
- Support the complete ticket lifecycle: create, assign, update and resolve.

- Help users find the current status of their reported incidents.
- Use configured AI FAQ responses to support common campus questions.
- Provide staff and administrators with useful ticket summaries.
- Deliver features incrementally using Agile and Scrum.

3.2 Expected Benefits

The proposed product is expected to improve transparency because users can refer to ticket status rather than repeatedly asking for updates. Centralized ticket history can help staff avoid missed requests and understand recurring incidents. Priority-based handling can make urgent issues more visible. These benefits depend on correct implementation, adoption and regular maintenance.

4. SOFTWARE PRODUCT ENGINEERING AND SDLC

Software product engineering considers the entire lifecycle rather than only writing code. A product team must understand users, select suitable technology, plan data storage, protect access, test the product and use feedback to improve it.

Stage	Application to this project	Output
1. Requirements analysis	Identify student, faculty, support staff and admin needs	Requirement list
2. Design thinking	Observe pain points and draft screen prototypes	Personas and wireframes
3. Architecture and data design	Plan frontend, APIs, ticket data and cloud services	Architecture and data model
4. Implementation	Build login, ticket, routing, FAQ and dashboard modules	Working modules
5. Testing and CI/CD	Check workflows, permissions and automated build	Test results and build pipeline
6. Deployment and feedback	Prepare cloud deployment and collect stakeholder feedback	Release candidate and improvement backlog

4.1 Programming vs. Product Engineering

Programming focuses on translating logic into executable code. Product engineering additionally considers usability, requirements, architecture, security, deployment, monitoring and the value delivered to users. For this project, a ticket form that compiles is not enough: it must validate information, store the ticket correctly, protect the user's data and help support staff complete their work.

5. IBM DESIGN THINKING AND USER RESEARCH

IBM Design Thinking is a human-centered approach. The Observe–Reflect–Make loop helps a team understand users, interpret their needs, create a possible solution and learn from feedback. The team should avoid assuming that a feature is useful until it has considered user needs and tested the design.

Stage	Application
Empathize	Understand how students, faculty and IT staff currently report and follow up on issues.
Define	State the core problem: reporting, assignment and status tracking are fragmented or manual.
Ideate	Consider ticket IDs, an FAQ assistant, category-based routing, notifications and dashboards.
Prototype	Sketch login, ticket creation, ticket status and admin dashboard screens.
Test	Ask representative users to try common tasks and provide feedback.

5.1 User Empathy Summary

User says / thinks	User does	User may feel
“How long will this ticket take?”	Emails staff or visits the IT office	Uncertain when there are no updates
“The Wi-Fi issue is urgent.”	Reports the issue and waits for help	Concerned about disruption to study
“Which issues are still pending?”	Checks multiple messages or asks staff	Frustrated by scattered information

The empathy summary is a planning aid. During real development, the team should validate these assumptions through interviews, observations or usability tests rather than treating them as measured research findings.

6. STAKEHOLDERS, PERSONAS AND ROLES

Stakeholder / Persona	Need	Project value
Student – Aniket	Report incidents and check status	Less uncertainty and easier follow-up
Faculty – Prof. Sharma	Flag a classroom projector problem as urgent	Teaching disruption can be reported clearly
IT Staff – Rajesh	Assign, prioritize and resolve many tickets	More organized workload
Administrator	Manage roles and view reports	Better operational oversight
Product Owner	Order requirements by value	Focus on the most useful work
Scrum Master	Facilitate Scrum and help remove impediments	Improved collaboration
Developers	Design, implement, test and document	A tested product increment

6.1 Role-Based Access Matrix

Capability	Student	Faculty	IT Staff	Admin
Ask configured FAQ	Yes	Yes	Yes	Yes
Create ticket	Yes	Yes	Yes	Yes
View own tickets	Yes	Yes	Yes	Yes
Assign / resolve tickets	No	No	Yes	Yes
Manage settings / reports	No	No	Limited	Yes

The matrix is a proposed access policy. The backend must enforce permissions on every protected operation; hiding a button in the interface alone is not sufficient security.

7. PRODUCT SCOPE AND EPICS

An Epic is a large area of functionality that can be split into smaller user stories. The following epics define the planned scope of the Smart Campus Helpdesk.

Epic	Scope
User and Role Management	Secure sign-in, user profile and role-aware access
AI FAQ Assistant	Answer configured questions such as Wi-Fi setup and password-reset guidance
Incident Management	Create, view, update, assign and close tickets
Routing and SLA Tracking	Support assignment by category and priority and identify overdue tickets
Notifications	Status-change alerts; email/SMS may be delivered in a later release
Dashboard and Reports	Display total, active, resolved and overdue ticket counts
Attachment Management	Store screenshot files and associate their references with tickets
Administration and Audit	Manage settings and review important system activity

8. USER STORIES AND ACCEPTANCE CRITERIA

A user story describes a requirement from the perspective of a person who will use the product. The common format is: “As a [user], I want [functionality], so that [business value].” Acceptance criteria are observable conditions that must be satisfied for the story to be accepted.

ID	User Story	Acceptance Criteria
US-01	As a campus user, I want to sign in securely so that I can access the helpdesk.	Valid credentials allow access; invalid credentials are rejected; role permissions apply.
US-02	As a student, I want to submit an incident so that support staff can investigate it.	Category, subject and description are required; unique ticket ID is generated; status begins as OPEN.
US-03	As a reporter, I want to view my ticket status so that I know its progress.	Own tickets are listed; status and priority are visible; unauthorized tickets are not exposed.
US-04	As a campus user, I want to ask an FAQ so that I can get quick guidance.	Configured FAQs return relevant answers; unsupported questions offer a staff/ticket fallback.
US-05	As IT staff, I want to assign a ticket so that one person is responsible for it.	Authorized staff can assign; assignment is saved; change is reflected in ticket details.
US-06	As IT staff, I want to prioritize incidents so that urgent issues are visible.	Priority is stored; routing follows defined rules; high-priority tickets are easy to identify.
US-07	As an administrator, I want ticket counts so that I can monitor workload.	Dashboard counts match stored ticket records and distinguish active/resolved/overdue.
US-08	As a reporter, I want to attach a screenshot so that staff can understand the issue.	Supported file uploads succeed; attachment links to the right ticket; errors are explained.

9. PRODUCT BACKLOG AND STORY POINTS

The Product Backlog is an ordered, evolving list of product work. The Product Owner orders items according to user value, urgency, dependencies, risk and feedback. The backlog is not a fixed list: items can be clarified, split, reprioritized or added as the team learns.

Order	Backlog Item	Priority	Points
1	Secure sign-in and role access	High	5
2	Create incident ticket	High	5
3	View own tickets and status	High	3
4	Ticket database connection	High	5
5	Assign ticket to IT staff	High	5
6	Configured AI FAQ assistant	High	8
7	Priority-based routing	High	5
8	Admin dashboard counts	Medium	5
9	Screenshot attachment support	Medium	5
10	Status-change notifications	Medium	5
11	Reports and ticket trends	Low	8
12	Predictive hardware analytics	Future	13

9.1 Story Point Estimation

Story points estimate relative effort, complexity and uncertainty. A Fibonacci-like sequence such as 1, 2, 3, 5, 8 and 13 can be used. For example, a basic ticket listing may be estimated at 3 points, while an AI FAQ integration may be estimated at 8 because it can involve configuration, service integration and testing. These estimates are illustrative and should be agreed by the team; they are not exact hours.

10. SPRINT PLANNING AND ROADMAP

A Sprint is a fixed-length period in which the team works toward a Sprint Goal and produces a usable increment. The following four-Sprint roadmap is a proposal; the team should adjust it to actual capacity, dependencies and available services.

Sprint 1 – Secure Foundation and Ticket Creation

Sprint Goal: Deliver a working foundation for secure access and basic incident reporting.

- Create project repository, README and standard folder structure.
- Define user roles and authentication flow.
- Design the initial user and ticket data model.
- Create the ticket form or API with required-field validation.
- Test basic login and ticket creation workflows.

Expected increment: initial login flow, ticket creation flow, basic data structure and documented test cases.

Sprint 2 – Ticket Tracking and Staff Workflow

Sprint Goal: Let users track incidents and allow support staff to manage assignments.

- Display a user's submitted tickets and their statuses.
- Add staff assignment and priority update functions.
- Implement initial category and priority routing rules.
- Demonstrate the workflow to stakeholders and capture feedback.

Sprint 3 – AI Assistance and Dashboard

Sprint Goal: Improve support speed and administrative visibility.

- Integrate the configured FAQ assistant, subject to service setup.
- Build dashboard counts for active, resolved and overdue tickets.
- Add attachment upload and ticket references.
- Review authorization, validation and failure messages.

Sprint 4 – Verification and Release Preparation

Sprint Goal: Validate the product and prepare a repeatable delivery process.

- Run functional tests and role-access tests.
- Configure a GitHub Actions workflow for automated build and tests.
- Prepare deployment configuration for IBM Cloud Code Engine.
- Finalize setup instructions, demonstration and acceptance review.

11. SCRUM BOARD AND DEFINITION OF DONE

A Scrum Board makes work visible to the whole team. A task moves to another column only when its state has actually changed. The board helps the team notice blocked work and avoid starting too many tasks at the same time.

TO DO	IN PROGRESS	CODE REVIEW	TESTING	DONE
Configure database	Ticket form	—	—	Repository setup
AI FAQ integration	—	—	—	Initial requirements
Dashboard counts	—	—	—	Backlog draft

The board above is an example for planning, not evidence that all listed tasks have been implemented. It should be updated during the actual Sprint.

11.1 Definition of Done

- Code follows the agreed project structure and naming conventions.
- Acceptance criteria for the story are satisfied.
- Relevant tests have passed.
- Authorization and input validation have been checked.
- Documentation is updated and changes are reviewed.
- The feature is integrated into the project without breaking existing functionality.

12. SCRUM EVENTS

12.1 Sprint Planning

The team confirms the Product Goal and Sprint Goal, reviews high-priority backlog items, clarifies acceptance criteria, estimates work, discusses dependencies and selects a realistic amount of work based on team capacity. The selected items form the Sprint Backlog.

12.2 Daily Scrum

Developers inspect progress toward the Sprint Goal and adapt their plan. The team may discuss: What was completed? What will be done next? Is anything blocked? The Daily Scrum is intended to improve coordination, not merely to provide a status report to a manager.

12.3 Sprint Review

At the end of the Sprint, the team demonstrates the working increment to stakeholders and discusses the outcome. For example, after ticket creation is demonstrated, a faculty member may request an urgency field. The Product Owner can record and prioritize that feedback in the Product Backlog.

12.4 Sprint Retrospective

The team inspects how it worked together, including communication, tools, quality and workflow. It identifies one or two practical improvements for the next Sprint, assigns responsibility and checks whether the change helped.

13. CONCEPTUAL SYSTEM ARCHITECTURE

The proposed system uses a layered architecture. The services below follow the earlier Unit 1 report. Their inclusion in the design does not mean that an external account, integration or deployment has already been configured.

Layer	Components	Responsibility
Users	Students, Faculty, IT Staff, Admin	Report incidents and manage support work
Frontend	Login, ticket form, status page, FAQ chat, dashboard	Provide user-facing screens
Backend APIs	Authentication, ticket workflow, routing, reporting	Validate requests and apply business rules
AI service	IBM Watson Assistant (planned)	Answer configured FAQs
Data layer	IBM Cloudant JSON documents (proposed)	Store ticket and user-related documents
File storage	IBM Cloud Object Storage (proposed)	Store screenshots and related files
Delivery	GitHub, GitHub Actions, IBM Cloud Code Engine	Version control, automated build/test and deployment

High-level flow: Campus User → Web Portal → Backend APIs → Ticket and Routing Services → Cloudant Database. FAQ requests may be sent to the Watson assistant. Screenshot files may be stored in object storage, with the file reference saved against the relevant ticket.

14. DATABASE DESIGN AND SAMPLE DATA

The data model separates users, tickets and attachment metadata. This separation supports clear ownership and makes it easier to manage tickets without storing every detail in a single oversized record.

Entity	Important Fields	Purpose
USERS	user_id, name, email, password_hash, role	Identity and access information
TICKETS	ticket_id, user_id, category, subject, description, status, priority, assigned_to, created_at	Incident and workflow information
ATTACHMENTS	attachment_id, ticket_id, file_url, uploaded_at	Attachment metadata and file reference

14.1 Sample Ticket Document

```
{
  "_id": "ticket::10482",
  "ticket_id": "INC-2026-0921",
  "category": "Network/Wi-Fi",
  "subject": "Unable to connect to campus Wi-Fi",
  "status": "IN_PROGRESS",
  "priority": "HIGH",
  "assigned_to": "TECH-STAFF-04"
}
```

The JSON is an illustrative example of a ticket document, not a live database record. Passwords must be stored as secure hashes, never as plain text. The backend should check ticket ownership and role permissions before returning or modifying records.

15. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system must do. Non-functional requirements describe quality attributes such as security, performance, availability and usability.

Type	Requirement	Description / Target
Functional	Authentication	Secure sign-in and role-aware access
Functional	AI FAQ	Answer configured Wi-Fi and password-reset questions
Functional	Ticket management	Create, view, update, assign and close tickets
Functional	Attachments	Upload screenshots and link them to tickets
Functional	Dashboard	Show total, active, resolved and overdue tickets
Non-functional	Security	HTTPS, secure password hashing and authorization checks
Non-functional	Performance	Proposed target: database queries under 500 ms under agreed test conditions
Non-functional	Availability	99.9% is a proposed target, not a measured result
Non-functional	Usability	Clear status labels, validation messages and simple navigation

16. TESTING AND ACCEPTANCE PLAN

Testing checks whether the product meets its acceptance criteria and whether changes have introduced defects. The team should test normal cases, invalid input, access permissions and failure conditions.

Test ID	Scenario	Expected Result
T01	Valid login	Authorized user signs in and sees permitted

		features
T02	Invalid login	Access is denied and a clear error is displayed
T03	Create ticket	Ticket is saved with a unique ID and OPEN status
T04	View another user's ticket	Unauthorized access is blocked
T05	Assign ticket	Authorized staff can assign a ticket and update priority
T06	FAQ question	Configured question receives a relevant answer
T07	Attachment upload	File reference is linked to the correct ticket
T08	Dashboard counts	Counts match underlying ticket records
T09	CI build and tests	Pipeline reports success only when configured checks pass

16.1 Acceptance Criteria for the Product

- Users can sign in and access only the functions permitted by their role.
- A ticket is stored with the required fields and a unique identifier.
- Users can view their own tickets and understand the current status.
- Configured FAQs return relevant answers or provide a fallback.
- Attachments are associated with the correct ticket.
- Dashboard figures match the ticket data used to calculate them.
- The configured automated build and test workflow runs successfully.

17. RISKS AND MITIGATION

Risk	Possible Impact	Mitigation
Unclear requirements	Rework and low user satisfaction	Use user stories, acceptance criteria and stakeholder feedback
AI response is unhelpful	User confusion	Limit initial scope to configured FAQs and provide a ticket/staff fallback
Unauthorized ticket access	Privacy and security risk	Enforce server-side role and ownership checks
Duplicate or incomplete reports	Extra support workload	Validate required fields and consider duplicate detection later
Cloud configuration problems	Delayed integration or deployment	Prototype connections early and keep credentials out of source control
Sprint overcommitment	Incomplete Sprint Goal	Select work based on capacity and dependencies

18. FUTURE SCOPE

- Email and SMS notifications for ticket status changes.
- Duplicate-ticket detection and knowledge-base recommendations.
- Predictive analytics for possible lab hardware failures.
- Mobile-friendly interface and accessibility improvements.
- Advanced SLA analytics and recurring-incident reports.
- If the product grows across multiple teams, consider scaling approaches such as LeSS, SAFe or Scrum@Scale.

19. CONCLUSION

The AI-Powered Smart Campus Helpdesk & Incident Management System proposes a structured approach to campus support requests. A single ticket workflow can improve reporting, assignment, status visibility and review

of recurring incidents. AI-assisted FAQs can support common questions, while the dashboard can help staff monitor workload.

The project combines the product concept, user-centered design and proposed cloud architecture from the earlier Unit 1 assignment with the Agile planning deliverables from the provided trainer guide. Scrum supports incremental delivery through Product Backlog refinement, Sprint Planning, Daily Scrum, Sprint Review and Sprint Retrospective. Planned integrations and tests must be implemented and verified before they are described as completed.

20. SUBMISSION DELIVERABLES

- Product Vision and stakeholder list.
- Epics, User Stories and Acceptance Criteria.
- Prioritized Product Backlog and Story Point estimates.
- Sprint Goals and Sprint Backlogs.
- Scrum Board and Scrum event notes.
- Conceptual architecture and data model.
- Testing and acceptance plan.
- Risk analysis, conclusion and future scope.
- Final demonstration or presentation.